

MATEMÁTICA COMPUTACIONAL APLICADA

Aula 12

Introdução a Grafos

Pontos e linhas para modelar qualquer relação — de redes sociais a mapas de estrada.

vértices

arestas

grau

ciclo



Nesta aula



Pares ordenados e relações

O mínimo necessário para entender grafos.



O que é um grafo

Vértices, arestas e os três tipos principais.



Representações

Lista e matriz de adjacência.



Grau, caminho e ciclo

Os conceitos que toda análise de grafo usa.



Pares ordenados e relações

Um par ordenado (a, b) registra dois elementos em ordem — diferente de conjuntos, aqui $(a, b) \neq (b, a)$, a menos que $a = b$.

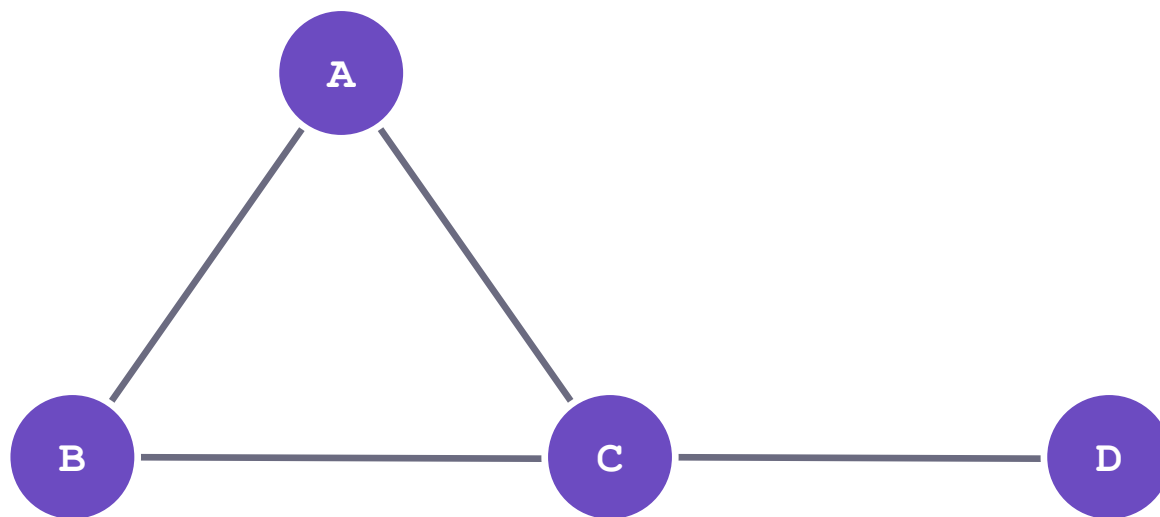
$$(a, b) \neq (b, a)$$

Uma relação entre elementos de um conjunto V é simplesmente um conjunto de pares ordenados (a, b) com $a, b \in V$.

Isso é exatamente o que precisamos para grafos: um grafo é uma relação (as arestas) sobre um conjunto (os vértices), desenhada como pontos e linhas em vez de escrita como pares.

O que é um grafo

Um grafo $G = (V, E)$ é um conjunto de vértices V e um conjunto de arestas E que os conectam.



$V = \{A, B, C, D\}$

$E = \{AB, AC, BC, CD\}$

É a estrutura ideal para modelar relações: redes sociais, mapas, dependências entre tarefas.

Três tipos de grafo



Não dirigido

Arestas sem direção: se $A-B$, então $B-A$.

ex.: amizades



Dirigido (dígrafo)

Arestas com seta: $A \rightarrow B$ não implica $B \rightarrow A$.

ex.: seguidores, links



Ponderado

Arestas têm peso (custo/distância).

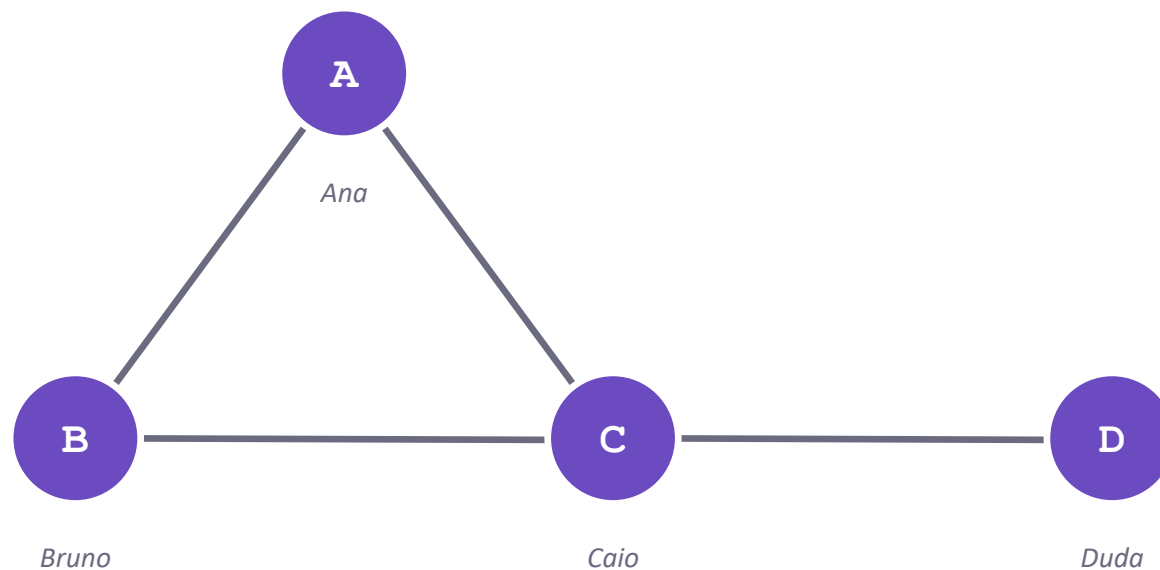
ex.: mapas rodoviários

EXEMPLO CONCRETO



Uma rede de amigos

Ana, Bruno, Caio e Duda numa rede não dirigida (amizade é mútua) — o mesmo grafo do Slide 4:



$E = \{\{Ana, Bruno\}, \{Ana, Caio\}, \{Bruno, Caio\}, \{Caio, Duda\}\}$ — a forma mais compacta de descrever um grafo.

EXEMPLO CONCRETO



Rotas entre cidades

Um mapa rodoviário é um grafo ponderado: vértices são cidades, arestas são estradas, peso é a distância (ilustrativo, em km):



SP → RP via Campinas: 95 + 120 = 215 km

EXEMPLO CONCRETO



Grafo dirigido é uma relação, literalmente

$V = \{A, B, C\}$, arestas $A \rightarrow B$ e $B \rightarrow C$:



$$R = \{ (A, B) , (B, C) \}$$

Um grafo não dirigido é uma relação simétrica: se $(A, B) \in R$, então $(B, A) \in R$ também.

Conceitos-chave

Grau de v

nº de arestas incidentes em v

Caminho

sequência de vértices ligados por arestas

Ciclo

caminho que começa e termina no mesmo vértice

Conexo

existe caminho entre quaisquer dois vértices



Handshake Lemma

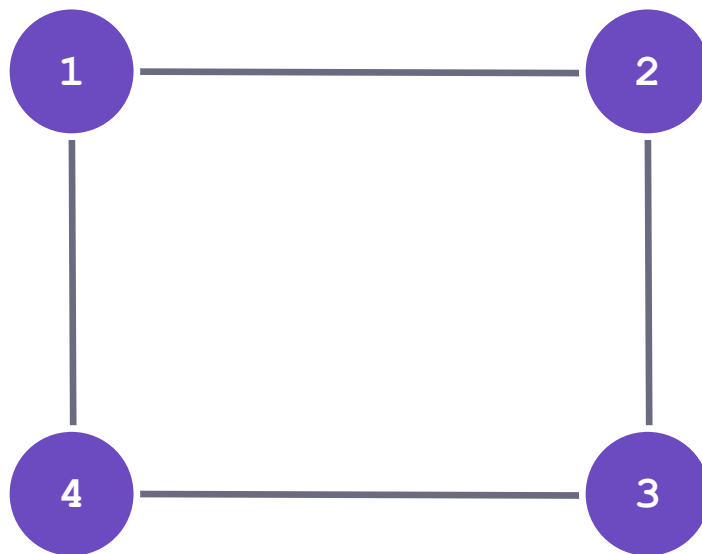
$$\sum \deg(v) = 2 \cdot |E|$$

A soma dos graus de todos os vértices é o dobro do número de arestas: cada aresta "toca" dois vértices, então é contada duas vezes.

CONFIRMANDO COM UM EXEMPLO

0 grau vem da matriz de adjacência

Num grafo não dirigido, o grau de um vértice é a soma da sua linha na matriz de adjacência.



```
V={1,2,3,4}
E={{1,2},{2,3},
   {3,4},{4,1}}
```

cada vértice tem grau 2 $\rightarrow \Sigma \deg(v) = 8 = 2 \cdot 4 \checkmark$

Lista de adjacência

Para cada vértice, a lista de vizinhos. Boa para grafos esparsos (poucas arestas).

Grafo.cs



```
var grafo = new Dictionary<string, List<string>> {  
    ["A"] = new List<string> { "B", "C" },  
    ["B"] = new List<string> { "A", "C" },  
    ["C"] = new List<string> { "A", "B", "D" },  
    ["D"] = new List<string> { "C" },  
};
```

grafo.py



```
grafo = {  
    "A": ["B", "C"],  
    "B": ["A", "C"],  
    "C": ["A", "B", "D"],  
    "D": ["C"],  
}
```

Matriz de adjacência

Matriz $n \times n$ onde $M_{ij} = 1$ se há aresta $i-j$. Boa para grafos densos (muitas arestas).

	A	B	C	D
A	0	1	1	0
B	1	0	1	0
C	1	1	0	1
D	0	0	1	0

Simétrica em torno da diagonal — sinal de que o grafo é não dirigido.



Testando propriedades

Grau e conectividade não precisam de desenho — dá para calcular direto a partir da lista de adjacência.

grau / Grau

O grau de um vértice é só o tamanho da sua lista de vizinhos.

Grafo.cs



```
static int Grau(  
    Dictionary<string, List<string>> g,  
    string v) => g[v].Count;
```

grafo_util.py



```
def grau(grafo, v):  
    return len(grafo[v])
```

```
print(grau(grafo, "C"))    # 3
```

eh_conexo / EhConexo

Uma busca a partir de um vértice: se alcançar todos os outros, o grafo é conexo.

Grafo.cs



```
static bool EhConexo(
    Dictionary<string, List<string>> g)
{
    var inicio = g.Keys.First();
    var visitados = new HashSet<string> { inicio };
    var fronteira = new Stack<string>();
    fronteira.Push(inicio);
    while (fronteira.Count > 0)
    {
        var v = fronteira.Pop();
        foreach (var viz in g[v])
            if (visitados.Add(viz))
                fronteira.Push(viz);
    }
    return visitados.Count == g.Count;
}
```

grafo_util.py



```
def eh_conexo(grafo):
    inicio = next(iter(grafo))
    visitados = {inicio}
    fronteira = [inicio]
    while fronteira:
        v = fronteira.pop()
        for vizinho in grafo[v]:
            if vizinho not in visitados:
                visitados.add(vizinho)
                fronteira.append(vizinho)
    return visitados == set(grafo)

print(eh_conexo(grafo)) # True
```




Onde grafos aparecem

Redes sociais (quem segue quem), mapas (cidades e estradas).

Dependências entre tarefas ou pacotes de software.

Redes de computadores, circuitos (Aula 09) e — como veremos — árvores (Aula 14), um caso especial de grafo.



Hora de praticar

Quatro exercícios: pares ordenados, matriz de adjacência, relação simétrica e um desafio sobre ciclos.



EXERCÍCIO 1

Grafo como pares ordenados

Escreva o grafo $V=\{1,2,3,4\}$, $E=\{\{1,2\},\{2,3\},\{3,4\},\{4,1\}\}$ como um conjunto de pares ordenados (trate como não dirigido, listando os dois sentidos). Ele é conexo? Tem ciclo?

$R =$ _____

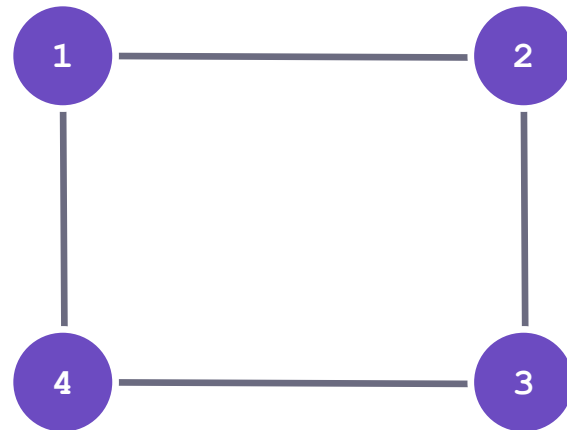
Conexo? _____ Tem ciclo? _____



GABARITO · EXERCÍCIO 1

Grafo como pares ordenados

$R = \{ (1,2) , (2,1) , (2,3) , (3,2) , (3,4) , (4,3) , (4,1) , (1,4) \}$



Sim, é conexo (dá para ir de qualquer vértice a qualquer outro). Sim, tem ciclo: 1-2-3-4-1 volta ao início.



EXERCÍCIO 2

Matriz de adjacência

Escreva a matriz de adjacência do grafo do Exercício 1 e some os graus. Confirme o Handshake Lemma.

	1	2	3	4
1	?	?	?	?
2	?	?	?	?
3	?	?	?	?
4	?	?	?	?



GABARITO · EXERCÍCIO 2

Matriz de adjacência

	1	2	3	4	grau
1	0	1	0	1	2
2	1	0	1	0	2
3	0	1	0	1	2
4	1	0	1	0	2

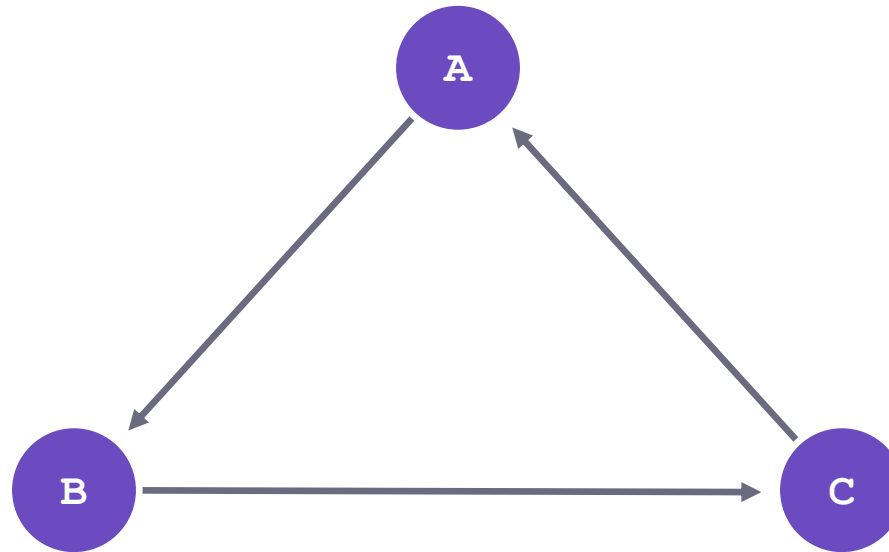
$$\sum \deg(v) = 2+2+2+2 = 8 = 2 \cdot 4 \text{ arestas } \checkmark$$



EXERCÍCIO 3

Relação simétrica?

Um grafo dirigido tem arestas $A \rightarrow B$, $B \rightarrow C$, $C \rightarrow A$. Ele representa uma relação simétrica? Justifique usando a definição de par ordenado.



Simétrica? _____ Justificativa: _____



GABARITO · EXERCÍCIO 3

Relação simétrica?

Não é simétrica.

$R = \{(A,B), (B,C), (C,A)\}$. Para ser simétrica, $(A,B) \in R$ exigiria $(B,A) \in R$ também — mas (B,A) não está na lista (só (B,C) está). Um par ordenado (a,b) é diferente de (b,a) , então essa troca não acontece "de graça".

Um ciclo dirigido geralmente NÃO é simétrico — simetria exigiria arestas nos dois sentidos entre cada par.



EXERCÍCIO 4 · DESAFIO

Detectando ciclos

Implemente `tem_ciclo(grafo)` para um grafo não dirigido em lista de adjacência (dica: uma busca que rejeita voltar direto para o vértice anterior, mas aceita revisitar por outro caminho).

```
tem_ciclo.py

def tem_ciclo(grafo):
    visitados = set()

    def dfs(v, pai):
        _____

    return _____
```

(percorre todo vértice ainda não visitado, chamando dfs)



GABARITO · EXERCÍCIO 4

Detectando ciclos

tem_ciclo.py



```
def tem_ciclo(grafo):  
    visitados = set()  
    def dfs(v, pai):  
        visitados.add(v)  
        for vizinho in grafo[v]:  
            if vizinho not in visitados:  
                if dfs(vizinho, v): return True  
            elif vizinho != pai:  
                return True  
        return False  
    for v in grafo:  
        if v not in visitados and dfs(v, None):  
            return True  
    return False
```

Testado no grafo do Slide 11 (o quadrado 1-2-3-4-1): retorna True.

PARA IR ALÉM

Referências

Livros — teoria



ROSEN, K. H.

Matemática Discreta e suas Aplicações. 7. ed.
AMGH/McGraw-Hill — cap. Grafos.



GERSTING, J. L.

Fundamentos Matemáticos para a Ciência da
Computação. 7. ed. LTC — cap. Grafos e Árvores.



GOODRICH, M. T.; TAMASSIA, R.

Estruturas de Dados e Algoritmos em Python.
Bookman — representação de grafos.

Documentação e vídeos



Python — NetworkX

[NetworkX — biblioteca de grafos](#) — tutorial oficial.



Vídeos

[Computerphile — introdução a grafos](#) — e suas representações.

[Matriz e lista de adjacência](#) — explicados com exemplos.



SÍNTESE

De pares ordenados a redes inteiras

**Toda relação vira grafo;
todo grafo esconde uma relação.**

Guarde grau, caminho, ciclo e conexo — esse vocabulário vale para o resto do curso, inclusive quando grafos virarem árvores.